# A Multi-Dimensional Analysis of LLM-Generated Python Code: Structural and Similarity Analysis with Reference Implementations and Functional Testing

***Technical Project Report***

Habtamu Tadesse Abebe (704241035)
Istanbul Technical University
Department of Computer Science

August 2026

# 1. Introduction

This report presents a multi-dimensional evaluation framework for assessing Large Language Model (LLM) generated Python code. The project investigates the structural, similarity, and functional characteristics of LLM-generated programs in comparison to reference implementations. By integrating similarity analysis, structural quality classification, and functional testing across 30 scenarios encompassing 142 distinct tasks, the project provides: (i) multi-dimensional similarity and structural analysis combined with prompt-derived functional testing; and (ii) an automated quality classifier using 12 static structural features and three supervised learning models Support Vector Machine (SVM), k-Nearest Neighbour (kNN), and Random Forest.

## 1.1 Research Objectives

The main objective of this project is to analyze and evaluate LLM generated Python code using structural, similarity-based, and functional testing approaches. The primary objectives of this study are:

1. To measure the lexical, structural, and semantic similarities between LLM-generated code and corresponding reference implementations.

2. To conduct a structural analysis of LLM-generated code to quantify its complexity.

3. To write and execute prompt specification-derived functional tests on both implementations to measure correctness.

4. To classify the structural quality of LLM-generated code using ML techniques based on static structural features.

# 2. Materials and Methods

The evaluation dataset encompasses 142 Python programming tasks from the InvisibleHand benchmark [1] distributed across 30 diverse application scenarios with up to five task variants per scenario. For each task, an LLM-generated implementation and at least one reference implementation were collected. The dataset comprises 284 LLM-generated and reference Python program pairs. Preprocessing involved tokenizing code streams and normalizing comments and whitespace to ensure fair comparison. A subset of 107 LLM code samples was explicitly labeled and isolated for training and evaluating the ML classifiers.

## 2.1 Similarity Analysis

Three similarity dimensions are computed per pair: lexical (TF-IDF cosine distance), structural (CFG-based graph comparison), and semantic (CodeBERT [2] embedding cosine distance), combined into a weighted final score:

$$S_{final} = 0.2\, S_{lexical} + 0.4\, S_{structural} + 0.4\, S_{semantic}$$

Where:

- $S_{lexical}$ = Lexical similarity score
- $S_{structural}$ = Structural similarity score
- $S_{semantic}$ = Semantic similarity score
- $S_{final}$ = Combined weighted final similarity score

## 2.2 Structural Analysis

Structural analysis extracts CFG metrics (cyclomatic complexity, nodes, edges, graph depth) and AST metrics (total nodes, depth, branching nodes, function/class definitions) per program. For quality classification, 12 static features per file - LOC, SLOC, average/max cyclomatic complexity, function/class count, max/avg nesting depth, comment

ratio, variable name length, unique variable count, and Halstead vocabulary - are labelled Good, Moderate, or Bad via a rule-based threshold scoring mechanism.

### 2.2.1 Machine Learning Models

Three classifiers (SVM, kNN, Random Forest) were trained to classify code quality using z-score-normalized features and evaluated using 5-fold cross-validation and a held-out test set of 107 samples.

## 2.3 Functional Testing

Functional test cases derived from original prompt specifications are executed on both LLM and reference programs. Both LLM-generated and reference implementations were subjected to identical test suites.

## 2.4 Dashboard Implementation

The project is supported by dedicated UI for presenting analysis results. First it uses a Streamlit for exploring similarity scores, structural metrics, and functional testing results across 30 scenarios and 284 program pairs. Second it uses a FastAPI-based web application with an HTML/CSS/JavaScript frontend, allowing users to upload Python source files and receive a quality classification (Good, Moderate, or Bad) together with the computed 12-dimensional feature vector. Some screenshots of the dashboard and classifier UI are provided in Appendix A.

# 3. Results

## 3.1 Similarity Analysis

Table 1 summarises similarity scores across 284 pairs. Semantic similarity dominates (mean = 0.961), reflecting consistent problem-domain intent regardless of implementation style. Lexical (mean = 0.159) and structural (mean = 0.154) scores are substantially lower. The weighted mean final Similarity Score S_final of 0.488 indicates moderate likeness; Figure 1 shows a right skewed distribution ranging from 0.367 to 0.771.

| Dimension | Mean | Std | Min | Max |
|---|---|---|---|---|
| Lexical | 0.159 | 0.111 | 0.000 | 0.643 |
| Structural | 0.154 | 0.193 | 0.000 | 0.640 |
| Semantic | 0.961 | 0.022 | 0.883 | 0.997 |
| Final (S_final) | 0.488 | 0.090 | 0.367 | 0.771 |

*Table 1. Descriptive statistics of similarity scores (n = 284 pairs).*

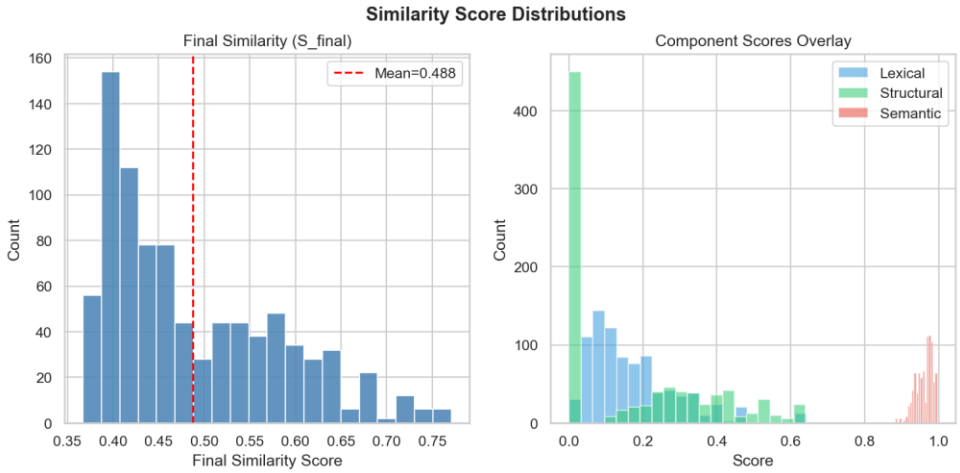



*Figure 1. S_final distribution (left, mean = 0.488) and component score overlay (right).*

Scenario-level variation is substantial. Authentication & Identity Management achieves the highest mean S_final (0.670), followed by Image Processing (0.626) and E-commerce (0.605). Voice & Video Communication (0.384), Video Streaming & Hosting (0.395), and Speech Recognition (0.409) rank lowest. The correlation heatmap (Figure 2) reveals that structural similarity dominates S_final (r = 0.96), while lexical (r = 0.51) and semantic (r = 0.49) components contribute moderately. S_final correlates near-zero with test pass rate (r = 0.06), confirming that structural similarity does not predict functional correctness.

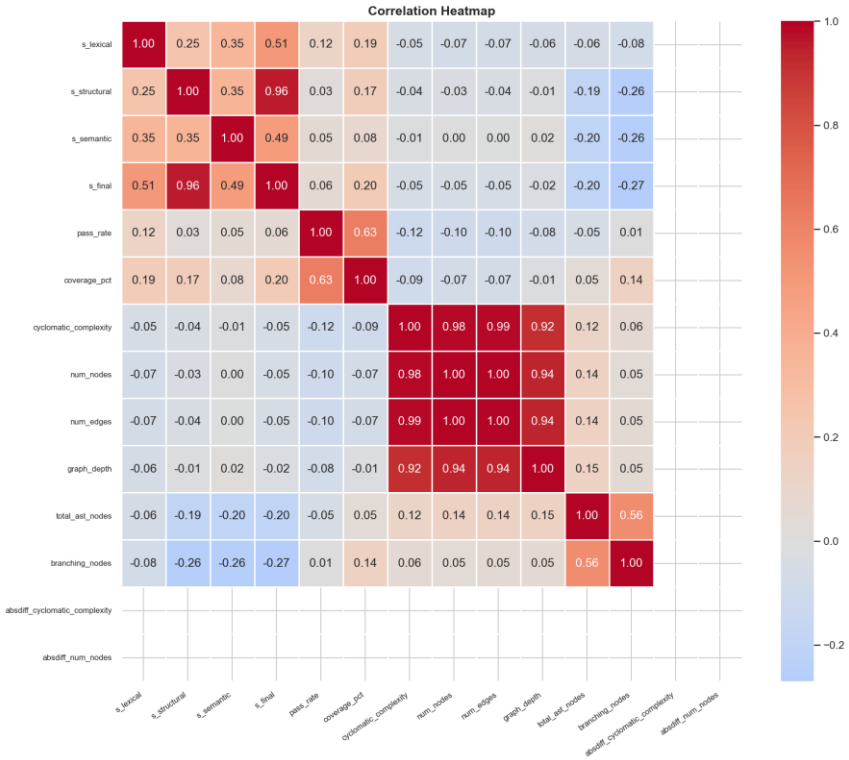



*Figure 2. Correlation heatmap. Key finding: S_final vs. pass_rate r = 0.06 (near-zero).*

## 3.2 Structural Analysis

LLM-generated programs are structurally more complex than reference implementations across all metrics (Table 2). Mean cyclomatic complexity is 2.08 vs. 1.19 (75% higher). LLM CFGs contain more than twice as many nodes (5.30 vs. 2.06) and AST total nodes (359 vs. 113), with substantially more branching nodes (7.48 vs. 1.11), indicating LLMs generate feature-complete, defensive implementations. Figures 3-4 show S_final decreases mildly with LLM complexity, and pass rate is independent of similarity.

| Structural Metric | LLM Mean | Reference Mean |
|---|---|---|
| CFG Cyclomatic Complexity | 2.08 | 1.19 |
| CFG Num Nodes | 5.30 | 2.06 |
| CFG Num Edges | 5.37 | 1.25 |
| CFG Graph Depth | 2.95 | 0.84 |
| AST Total Nodes | 359.4 | 112.9 |
| AST Branching Nodes | 7.48 | 1.11 |
| AST Function Definitions | 4.25 | 1.07 |

*Table 2. Mean structural metrics: LLM vs. reference implementations (n = 284).*

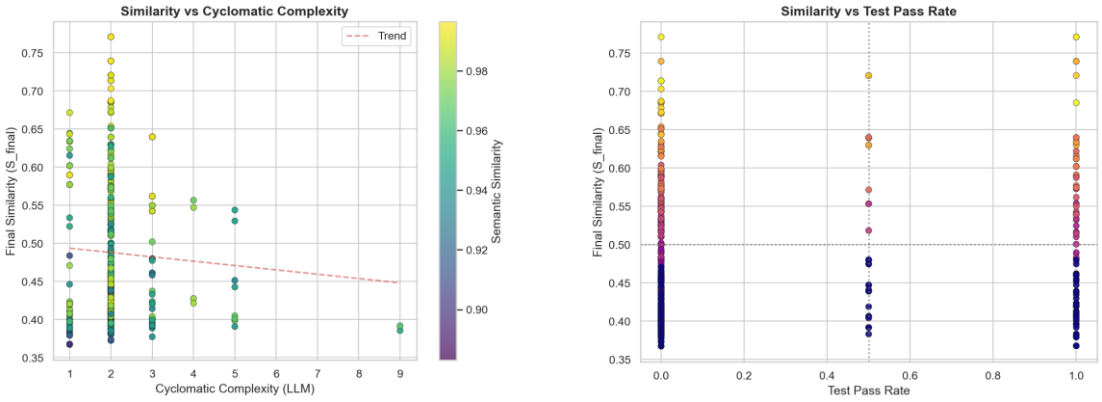



*Figure 3 (left): S_final vs. LLM cyclomatic complexity. Figure 4 (right): S_final vs. test pass rate (r = 0.06).*

### 3.2.1 Structural Quality Classification

The structural quality of the LLM-generated code containing a total of 726 code samples generated by GPT-based models using prompts derived from the InvisibleHand dataset across multiple execution iterations was evaluated and labeled using a rule-based multi-feature scoring. The resulting distribution showed a strong tendency toward high structural quality, with 59.5% (432 samples) classified as 'Good' and 32.2% (234 samples) as 'Moderate'. Only 6.3% (46 samples) were classified as 'Bad', while 1.9% (14 samples) remained 'Unclassified' due to parsing constraints (Figure 5).

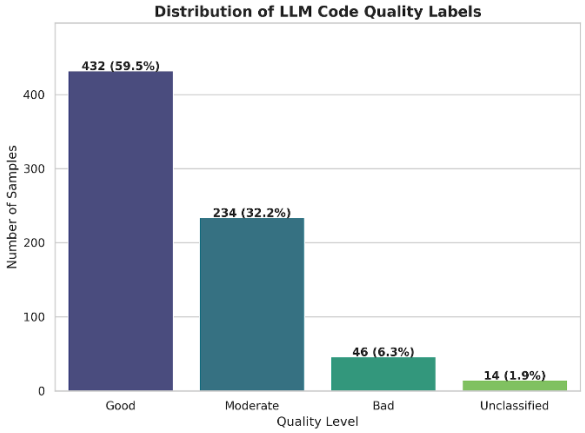



*Figure 5. Distribution of LLM code quality labels*

Table 3 presents classifier performance on the 107-sample test set. SVM achieves best results - 97.2% accuracy, macro F1 = 0.955, ROC-AUC = 0.999. Random Forest ranks second (92.5%, AUC = 0.989). kNN is weakest (88.8%, AUC = 0.933). CV accuracy is consistent with test accuracy, confirming stable generalization.

| Model | Test Acc | Prec (Mac) | Rec (Mac) | F1 (Mac) | ROC-AUC | CV Acc | CV Std |
|---|---|---|---|---|---|---|---|
| SVM | 0.972 | 0.980 | 0.933 | 0.955 | 0.999 | 0.976 | 0.014 |
| Random Forest | 0.925 | 0.878 | 0.818 | 0.842 | 0.989 | 0.898 | 0.012 |
| kNN | 0.888 | 0.833 | 0.747 | 0.775 | 0.933 | 0.876 | 0.024 |

*Table 3. Classifier performance (n = 107). SVM row highlighted. Mac = macro-averaged.*

Confusion matrices (Figure 6) confirm SVM superiority: only 3 misclassifications vs. 8 for Random Forest and 12 for kNN. ROC curves (Figure 7) show SVM AUC = 1.000 for Bad and Good classes; kNN degrades on Bad (AUC = 0.846).

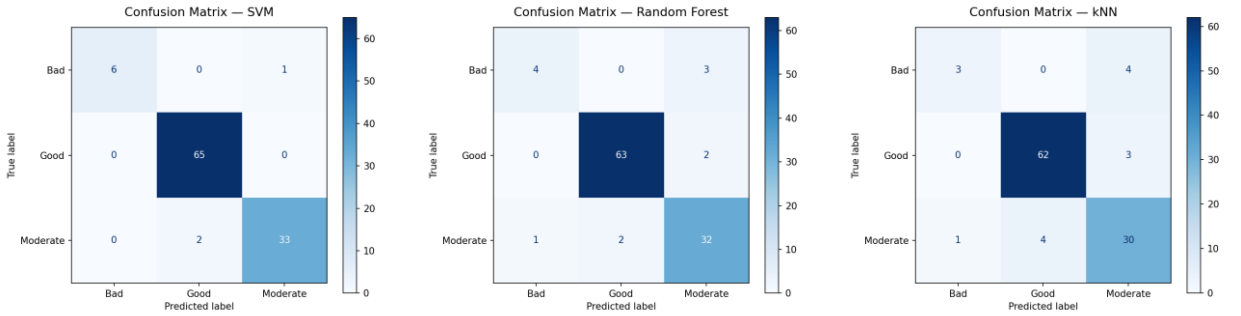



*Figure 6. Confusion matrices: SVM (left), Random Forest (centre), kNN (right).*

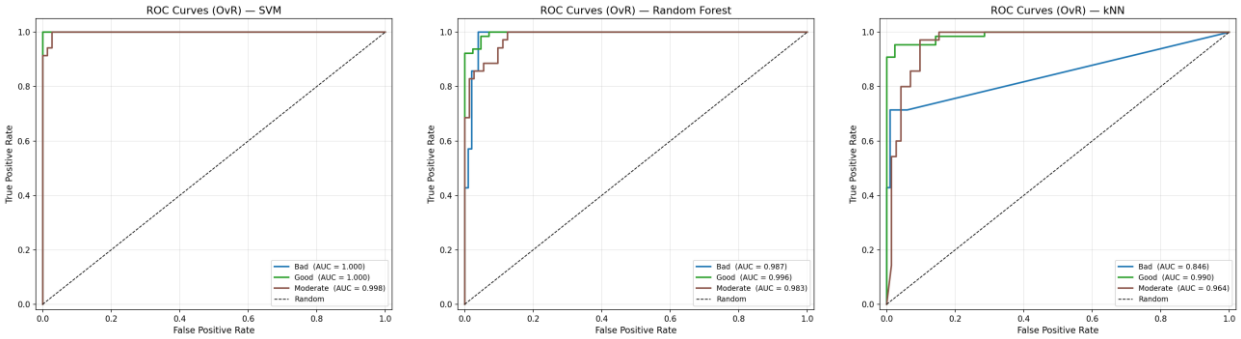



*Figure 7. ROC curves (one-vs-rest): SVM (left), Random Forest (centre), kNN (right).*

## 3.3 Functional Testing

Functional tests were executed on 38 LLM and 76 reference program (two reference programs per task) records. LLM programs achieve a mean pass rate of 90.8% (34/38 tests passed) vs. 36.8% for references (28/76). This does not imply LLM superiority but; reference programs tend to be minimal implementations not fully satisfying the complete specification operationalized in the test cases. Detailed analysis of these test results revealed distinct failure patterns. The reference implementations frequently failed because developers submitted incomplete conceptual stubs, ignored core prompt constraints (e.g., utilizing irrelevant third-party API boilerplate instead of building requested cryptographic logic), or triggered fatal global-scope execution crashes upon import. In contrast, the LLM consistently generated complete, runnable scripts protected by safe execution wrappers.

| Variant | Records | Mean Pass Rate | Tests Passed | Total Tests |
|---|---|---|---|---|
| LLM-Generated | 38 | 0.908 | 34 | 38 |
| Reference (ref_1 + ref_2) | 76 | 0.368 | 28 | 76 |

*Table 4. Aggregate functional testing results.*

Scenario-level results (Table 5) show notable divergence. The LLM achieves 1.000 pass rate in Blockchain & Cryptocurrency while the reference scores 0.000, confirming structural resemblance is neither necessary nor sufficient for functional correctness. Conversely, in Video Streaming & Hosting the reference passes all tests while the LLM achieves only 0.500.

| Scenario | LLM Pass Rate | Ref Pass Rate |
|---|---|---|
| Authentication & Identity Management | 0.333 | 0.333 |
| Blockchain & Cryptocurrency | 1.000 | 0.000 |

| | | |
|---|---|---|
| Data Analysis | 0.750 | 0.000 |
| E-commerce | 1.000 | 0.500 |
| Image Processing | 1.000 | 1.000 |
| Machine Learning - AI Deployment | 1.000 | 0.000 |
| Natural Language Processing | 1.000 | 1.000 |
| Payment Processing | 1.000 | 0.500 |
| Speech Recognition | 1.000 | 0.500 |
| Video Streaming & Hosting | 0.500 | 1.000 |
| Weather Data | 1.000 | 0.500 |

*Table 5. Selected scenario-level functional test pass rates.*

## 4. Discussion and Conclusion

The findings conclusively demonstrate that LLM-generated Python code exhibits both acceptable structural quality and reliable functional correctness. Structurally, the generated implementations are highly viable, with over 91% of the evaluated samples categorized as 'Good' (59.5%) or 'Moderate' (32.2%) by the static multi-feature scoring mechanism. Functionally, the AI-generated code proved reliable, achieving a 90.8% test pass rate.

Results indicate that while LLM-generated code exhibits low lexical (0.158) and structural (0.154) similarity to reference code, it achieves very high semantic similarity (0.961) and significantly outperforms reference code in functional testing (90.8% vs. 36.8% pass rate). This indicates that low textual/structural similarity does not imply poor quality or incorrectness; rather, the LLM has functionally correct algorithms that require larger, more complex abstract syntax trees (359 vs 113 nodes), higher cyclomatic complexity (2.08 vs. 1.19), and more branching (7.48 vs. 1.11 nodes). Furthermore, static structural features successfully classify code quality, with Support Vector Machine (SVM) achieving 97.20% accuracy on a 107-sample dataset; however, structural quality does not guarantee functional correctness, and the classification framework should complement rather than replace functional testing. The low reference pass rate (36.8%) reflects the minimal nature of reference implementations rather than LLM superiority. Future work should extend this framework to additional LLM families, validate quality labels against expert review, and investigate prompt design effects on structural and functional program properties.

## 5. Limitations

- Dataset scope: The dataset covers 30 scenarios from the InvisibleHand benchmark. Results may not generalize to other programming domains, languages, or LLM families not included in the study.
- Reference quality: Reference implementations are not necessarily production-quality. Their low pass rate on specification-derived tests suggests they represent minimal implementations rather than comprehensive solutions.
- Quality labelling: The rule-based quality labelling procedure uses community accepted threshold-based rules without external validation against human expert judgements. The thresholds may not generalize across programming domains.
- Test coverage: Tests are designed from the natural-language specification and may not cover all edge cases. Pass/fail outcomes represent empirical evidence for the tested inputs only, not exhaustive correctness proofs.

## References


[1] Xiaoyu Zhang, Juan Zhai, Shiqing Ma, Qingshuang Bao, Weipeng Jiang, Qian Wang, Chao Shen, & Yang Liu. (2025). The Invisible Hand: Unveiling Provider Bias in Large Language Models for Code Generation. Proceedings of the Association for Computational Linguistics (ACL).

[2] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong., Shou, L., Qin, B., Liu, T., & Zhou, M. CodeBERT: A pre-trained model for programming and natural language. Findings of EMNLP, 2020, 1536–1547.

[3] T. J. McCabe, "A Complexity Measure," IEEE Trans. Softw. Eng., vol. SE-2, no. 4, pp. 308-320, 1976.

[4] Zhang, K., & Shasha, D. (1989). Simple fast algorithms for the editing distance between trees and related problems. SIAM Journal on Computing, 18(6), 1245–1262.

[5] Allamanis, M., Barr, E. T., Devanbu, P., & Sutton, C. (2018). A survey of machine learning for big code and naturalness. ACM Computing Surveys, 51(4), 81:1–81:37.

[6] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. O., Kaplan, J., & Zaremba, W. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

[7] Fenton, N., & Bieman, J. (2014). Software Metrics: A Rigorous and Practical Approach (3rd ed.). CRC Press.

[8] C. Cortes and V. Vapnik, "Support-Vector Networks," Mach. Learn., vol. 20, no. 3, pp. 273-297, 1995.

[9] L. Breiman, "Random Forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001.

[10] T. Cover and P. Hart, "Nearest Neighbor Pattern Classification," IEEE Trans. Inf. Theory, vol. 13, no. 1, pp. 21-27, 1967.

[11] Halstead, M. H. (1977). Elements of Software Science. Elsevier North-Holland.

[12] Lessmann, S., Baesens, B., Mues, C., & Pietsch, S. (2008). Benchmarking classification models for software defect prediction: A proposed framework and novel findings. IEEE Transactions on Software Engineering, 34(4), 485–496.


## Appendix A — Additional UI Dashboard Screenshots

Figure A.1. Main dashboard interface for similarity, structural and functional testing evaluation.

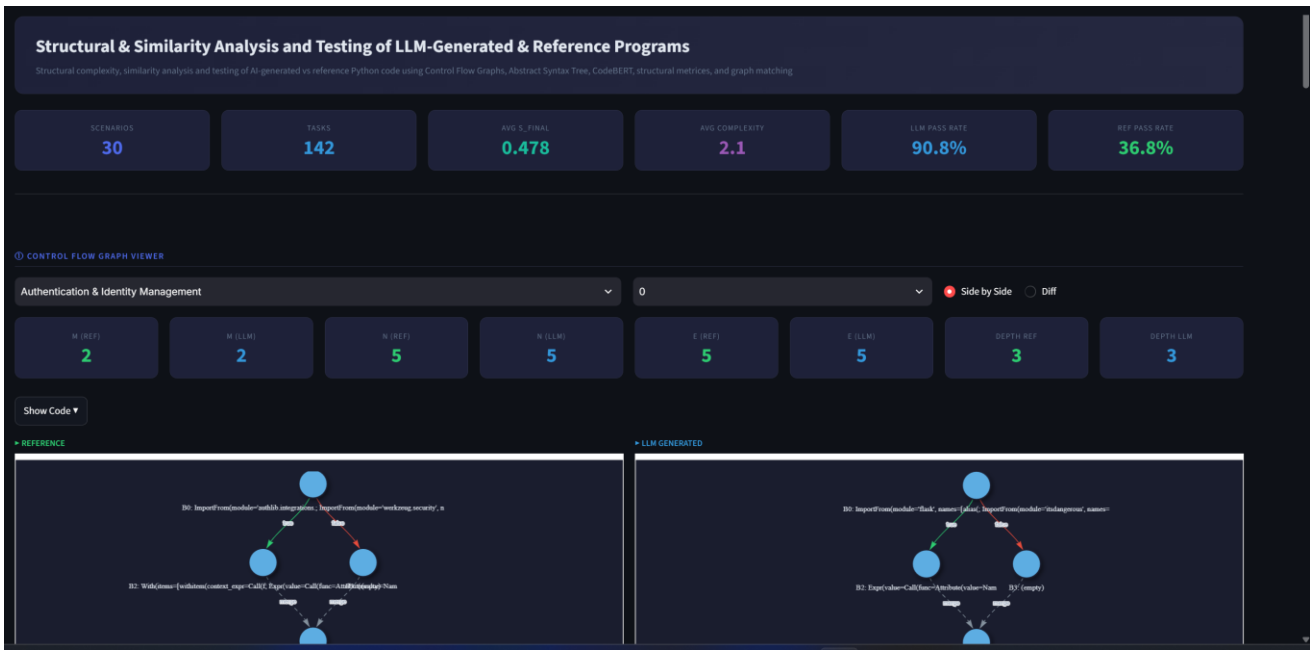


Figure A.2. Similarity analysis results by scenario.




| Scenario | Tasks | Mean S_final | Mean Lexical | Mean Semantic | Mean Structural |
|---|---|---|---|---|---|
| Authentication & Identity Management | 10 | 0.668 | 0.317 | 0.987 | 0.525 |
| Blockchain & Cryptocurrency | 10 | 0.558 | 0.108 | 0.982 | 0.357 |
| Cloud Database Services | 10 | 0.426 | 0.178 | 0.976 | 0.000 |
| Cloud Hosting | 10 | 0.487 | 0.172 | 0.946 | 0.186 |
| Container Orchestration | 10 | 0.576 | 0.321 | 0.967 | 0.312 |
| Content Moderation & Filtering | 10 | 0.425 | 0.084 | 0.964 | 0.056 |
| Data Analysis | 10 | 0.432 | 0.148 | 0.955 | 0.052 |
| Data Storage | 10 | 0.429 | 0.123 | 0.939 | 0.071 |
| Data Visualization | 10 | 0.431 | 0.141 | 0.960 | 0.046 |
| E-commerce | 10 | 0.593 | 0.205 | 0.981 | 0.398 |

Figure A.3. Main UI for code quality classifier

